SOW Pilot AI

Technical Architecture & Demo Presentation Guide

Agentic Bid & Proposal Automation Platform
October 2026

1. Executive Summary

SOW Pilot AI automates the end-to-end lifecycle of responding to a Statement of Work (SOW) or RFP — from the moment a solicitation document arrives, through bid qualification, cost and resource planning, client-facing proposal generation, internal governance, sign-off, client negotiation, contract tracking, and recording the outcome once the bid is decided.

The platform is built around a core processing pipeline of eight specialized AI agents, each responsible for one stage of analysis (requirements, resourcing, risk, architecture, timeline, cost, internal readiness, and client-facing proposal writing), supplemented by a set of on-demand agents triggered at specific points in the lifecycle (qualification, pre-bid clarification, internal approvals, worker matching). Deterministic calculation is used wherever a number must be trustworthy — cost estimation and bid qualification are computed from real data (the organization's actual worker billing rates, actual engaged weeks per role, and a configured margin), not inferred by the language model.

This document covers the system architecture, the complete technology stack, the role of each agent in the pipeline, and a screen-by-screen guide for presenting the platform's lifecycle in a live demonstration.

2. System Architecture & Workflow

The diagram below traces one SOW from intake through to its final post-award outcome. Stages 1–4 run automatically and unattended; stages 5, 6b, 7, and 9 are points where a human decision changes what the system does next — these are not simply sequential steps but checkpoints with real gating logic.

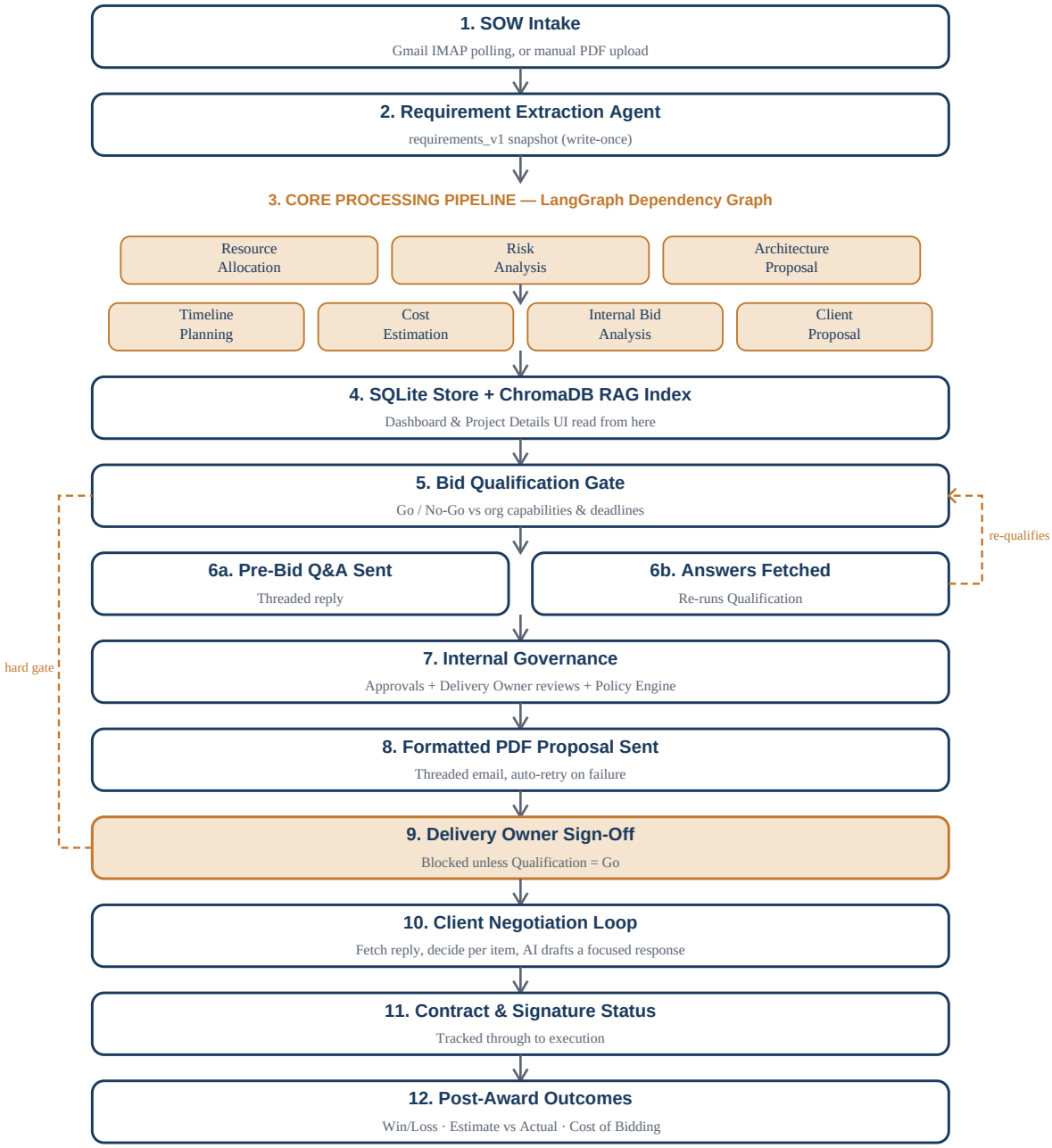

Figure 1 — End-to-end SOW lifecycle, from inbound document to post-award outcome tracking. Dashed lines indicate feedback loops where a later decision re-triggers an earlier stage.

2.1 Stage-by-Stage Description

Stage	What Happens
1. SOW Intake	A SOW arrives as an email attachment (polled from Gmail via IMAP every 30 seconds) or is uploaded manually. No manual re-typing of content occurs at any point downstream.
2. Requirement Extraction	The Requirement Extraction Agent parses the raw PDF text into a structured, write-once requirements snapshot (requirements_v1): project scope, technology stack, deliverables, deadlines, required certifications, evaluation criteria, and required bid documents.
3. Core Processing Pipeline	Seven further agents run as a LangGraph dependency graph: Resource Allocation, Risk Analysis, and Architecture Proposal execute in parallel (all depend only on the extracted requirements); Timeline Planning then depends on the resourcing output; Cost Estimation depends on resourcing, timeline, and risk; Internal Bid Analysis and Client Proposal run last, synthesizing everything produced before them.
4. Persistent Store	Results are written to a SQLite database (the system of record) and indexed into a ChromaDB vector store for retrieval-augmented features. The Dashboard and Project Details UI read live from this store.
5. Bid Qualification Gate	A deterministic Go/No-Go check compares the SOW's required certifications and submission deadline against the organization's actual capability list. A missing certification can be exempted by a reviewer, which recalculates the decision immediately.
6. Client Clarification Loop	Clarifying questions are sent to the client as a threaded reply to the original inbound email. When the client replies in that same thread, an agent matches the reply text to the specific pending questions, saves the answers, and automatically re-runs the Qualification check against the updated facts.
7. Internal Governance	Internal concerns (pricing risk, staffing gaps, compliance flags) are routed for approval; the assigned Delivery Owner reviews the project at the Solutioning and Pricing stages — not only at the end; and a separate, rule-based Policy Compliance Engine independently checks budget thresholds, resource/skill alignment, security staffing, and rate-card limits.
8. Proposal Delivery	The Client Proposal Agent's persuasive narrative — grounded in the organization's pre-approved Content Library for brand consistency — is rendered into a formatted PDF (investment table, delivery schedule table, bulleted differentiators) and sent as a short cover email with that PDF attached, threaded into the original conversation.
9. Delivery Owner Sign-Off	The final submission-ready approval is hard-gated: it cannot be marked Approved unless the Qualification decision is 'Go' and all internal approvals are resolved, preventing a bid from reaching 'ready to submit' without having actually cleared qualification.
10. Client Negotiation Loop	The client's reply to the sent proposal is read directly from the inbox and classified as Accepted, Rejected, or Revision Requested, with every distinct ask broken out individually. A human decides Accept / Decline / Partial on each one; an Accepted cost change is applied immediately to the real bid price (see Section 4.2), and an AI agent drafts a short, focused reply addressing every point — never a full proposal re-send. Every round is logged to a version history.

11. Contract & Signature Status	Once terms are agreed, the status of the actual contract/SOW signature process is tracked (Not Started / Sent for Signature / Signed / Rejected) against the project.
12. Post-Award Outcomes	After the client's decision is known, the Win/Loss result, the actual delivered cost and duration (compared against the original estimate), and the internal hours spent preparing the bid are recorded against the project, closing the loop between what was bid and what actually happened.

3. Technology Stack

The platform is built entirely on open-source and standard cloud-SDK tooling, with one managed AI service (AWS Bedrock) as the reasoning engine behind every agent.

3.1 Frontend

Tool	Purpose
React 18 + TypeScript	Component-based UI framework with static typing across the application.
Vite	Development server and production build tool.
Tailwind CSS	Utility-first styling framework for all layout and visual design.
Recharts	Charting library used for budget allocation and dashboard visualizations.
Framer Motion	Animation library for UI transitions.
lucide-react	Icon set used throughout the interface.
Axios / Fetch	HTTP communication with the backend API.

3.2 Backend & Orchestration

Tool	Purpose
FastAPI	Python web framework exposing all REST and Server-Sent Event (SSE) endpoints.
Uvicorn	ASGI server running the FastAPI application.
Pydantic	Request/response schema validation for every API endpoint.
LangGraph	Builds the dependency-aware execution graph for the eight-agent core pipeline — agents with no interdependency (Resource Allocation, Risk Analysis, Architecture Proposal) run in parallel; agents that depend on prior output (Timeline, Cost, Internal Analysis, Proposal) run only after their dependencies complete.
SQLite	System-of-record database for all project, qualification, approval, and outcome data.
ChromaDB	Vector database powering retrieval-augmented (RAG) features over historical SOW requirements.

3.3 AI & Document Processing

Tool	Purpose
AWS Bedrock — Claude Haiku 4.5	Primary reasoning engine for all LLM-driven agents (extraction, resourcing, risk, architecture, timeline, internal analysis, client proposal, qualification support), accessed via the boto3 Converse API.

Google Gemini Embeddings API	Generates vector embeddings for the RAG retriever (text-embedding-04), with automatic fallback to ChromaDB's local embedding model if no Gemini key is configured — keeping the RAG feature functional offline.
pypdf	Extracts raw text from inbound SOW PDF documents.
reportlab	Generates the formatted client proposal PDF (investment table, delivery timeline table, bulleted differentiators) sent to the client.
python-docx	Supports Word-format document handling where required.

3.4 Email Integration

Gmail is integrated directly via the Python standard library's imaplib (inbound polling, reply matching) and smtplib (outbound sending), rather than a third-party email service. Every outbound message sets its own Message-ID and, where applicable, In-Reply-To/References headers, so replies thread correctly in the client's inbox exactly as if a person had replied manually.

4. Core Processing Pipeline — Eight Agents

These eight agents run automatically, in dependency order, every time a SOW is ingested. Cost Estimation is explicitly deterministic — it does not ask the language model to produce a dollar figure; it calculates one from real worker billing rates and real engaged-week data.

Agent	Reasoning	Role
1. Requirement Extraction	LLM	Parses the raw SOW text into structured requirements: scope, tech stack, deliverables, deadlines, required certifications, evaluation criteria, and required bid documents.
2. Resource Allocation	LLM	Recommends the delivery team composition (roles and headcount) based on the extracted technology stack and scope.
3. Risk Analysis	LLM	Identifies delivery risks with probability and impact ratings, which feed directly into the deterministic contingency calculation in Cost Estimation.
4. Architecture Proposal	LLM	Designs the proposed technical solution and component architecture.
5. Timeline Planning	LLM	Builds the phased delivery schedule, including a per-role engagement window (start/end week) — not every role is engaged for the full project duration.
6. Cost Estimation	Deterministic	Computes the full budget from the organization's real Workers directory billing rates, each role's actual engaged weeks (from Timeline Planning), risk-weighted contingency, and a configurable margin — no cost figure is guessed by the model.
7. Internal Bid Analysis	LLM + Deterministic	Synthesizes an internal-only readiness report (executive summary, readiness score, recommendations) from the outputs already produced by agents 1–6.
8. Client Proposal	LLM	Writes the persuasive, client-facing proposal — weighted to the client's stated evaluation criteria and grounded in the organization's pre-approved Content Library for consistent brand messaging.

4.1 Supporting On-Demand Agents

The following agents are not part of the automatic ingestion pipeline — each is triggered on demand, at a specific point in the lifecycle, by a user action.

- **Bid Qualification Agent** — deterministic Go/No-Go check, triggered from the Qualification tab.
- **Pre-Bid Clarifying Questions Agent** — drafts client-facing clarification questions, triggered from the Pre-Bid Q&A tab.
- **Client Answer Matching Agent** — matches an inbound email reply to the specific pending questions it answers.
- **Certification Exemption Detector** — scans answered Q&A for language granting an exemption on a missing certification.
- **Internal Approval Concerns Agent** — flags items needing internal sign-off, triggered from the Approvals tab.
- **Worker-to-Task Matching Agent** — matches a specific open task to the best-fit available worker, scored against their real role and skills.

- **Policy & Compliance Guardrail Engine** — a separate, rule-based (non-LLM) system that independently checks budget thresholds, resource/skill alignment, security staffing, and rate-card limits on every ingested project.
- **Client Response Classification Agent** — reads a client's reply to a sent proposal and classifies it as Accepted, Rejected, or Revision Requested, breaking out each distinct request raised.
- **Focused Reply Drafting Agent** — once a human has decided Accept/Decline/Partial on every client request, drafts a short, direct reply addressing each point — not a restatement of the whole proposal.

4.2 Deterministic Price Adjustment Architecture

Any time a number changes — a Finance-approved payment-terms premium, or a client-negotiated discount — the platform uses one single mechanism, not a different ad-hoc calculation per screen. This matters because a persuasive document (an email, a proposal narrative) and a financial record (the bid price shown in the PDF, the Dashboard, the budget breakdown) must never disagree with each other.

- **One source of truth.** A single function applies every price change — whether it originates from an internal stakeholder approving a concern on the Approvals tab, or from a human deciding to accept a client's cost request during negotiation. Both paths call the same deterministic calculation against the same stored bid price.
- **Human decides the percentage; the system does the arithmetic.** The AI never invents a discount or a new total on its own — a person enters the agreed percentage, and the delta is computed directly from the real stored number.
- **Fully traceable.** Every adjustment is appended to a permanent, timestamped log on the project (who/what triggered it, the percentage, the resulting dollar delta, the stated reason) — never overwritten, so the final price can always be explained back to its origin.
- **Propagates everywhere automatically, with no extra wiring.** Because the PDF's investment table, the Resources tab budget dashboard, and the Dashboard project card all read the same stored field, one write updates every surface at once — there is no separate step to 'sync' the number anywhere.
- **The AI is handed the real number, not asked to produce one.** When the Focused Reply Drafting Agent confirms a price change to the client, it is given the exact figure the system already calculated and instructed to state it verbatim — removing any possibility of the email and the financial record disagreeing.

5. Demo Presentation Guide

The following sequence presents the platform as one continuous story — a single SOW moving through the full lifecycle — rather than a feature-by-feature tour. Each entry names the screen, what it displays, and what it demonstrates about the lifecycle.

Dashboard

Project cards + live stats	Shows volume processed, budgets, and readiness scores across all ingested projects — establishes that this is a running system with real history, not a single demo project.
Scan Gmail action	Triggers a live IMAP poll — demonstrates that inbound SOWs are picked up automatically, with no manual re-entry.

Processing (live ingestion)

SSE live agent panel	Displays all eight core agents executing in sequence/parallel in real time — the single clearest visual proof that the system is doing real, traceable work rather than returning a canned result.

Project Details — Overview

Executive summary, tech stack, deliverables	Confirms everything shown was extracted from the source SOW, not hand-entered.

Project Details — Qualification

Go/No-Go decision + reasoning	Shows the deterministic qualification check and how an exemption flips the decision live — the clearest proof of reasoning over real facts rather than static output.
Required Bid Documents checklist	Tracks supporting documents (reference letters, financials, etc.) distinct from certifications.
Delivery Owner assignment	Shows the delivery owner engaged from qualification time, not only at final sign-off.
Policy & Compliance panel	Shows the independent, rule-based guardrail engine running alongside the LLM-driven qualification — a second layer of validation.

Project Details — Pre-Bid Q&A

Generate & send clarifying questions	Shows the question thread as a reply in the same email the SOW arrived on.
Fetch Answers from Mail	Demonstrates the client-reply matching and automatic re-qualification — the strongest proof that the system reasons over new information rather than just storing text.

Project Details — Resources

Team roles with engagement week ranges	Shows that costing is based on real per-role engagement windows, not a flat rate across the whole project duration.
Delivery Owner review — Solutioning checkpoint	Shows ongoing delivery-owner involvement, not only a bookend review.

Project Details — Proposal

Generated persuasive proposal	Shows the client-facing narrative, weighted to the client's stated evaluation criteria.
Send Proposal to Client + PDF preview	Shows the short cover email and the formatted PDF attachment — investment table, delivery schedule table, bulleted differentiators — the final deliverable the whole pipeline exists to produce.
Fetch Client Response from Mail	Reads the client's real reply and classifies it — Accepted, Rejected, or Revision Requested, with each distinct ask broken out individually.
Decide on Each Request + Draft Reply with AI	A human decides Accept/Decline/Partial per item, optionally applying a real price adjustment; the AI drafts a short, focused reply — the strongest proof that negotiation is tracked, not lost, and that the financial record and the email can never disagree.

Project Details — Approvals

Internal concerns + resolution	Shows internal governance routing before anything reaches the client.
Delivery Owner review — Pricing checkpoint	Second ongoing delivery-owner checkpoint, before final sign-off.
Delivery Owner Sign-Off (qualification-gated)	Attempting to approve before Qualification is 'Go' is blocked with a clear message — proves the system enforces correct sequencing, not just a UI that allows any action.

Project Details — Outcomes

Contract & Signature Status	Tracks where the actual signature process stands, once terms are agreed.
Win/Loss, Estimate vs. Actual, Cost of Bidding	Closes the lifecycle: shows what happened after the proposal was sent, including a real cost overrun example and a loss example — proof the system is presented honestly, not only showing favorable outcomes.

Settings

Content Library	Shows the mechanism keeping every generated proposal consistent with pre-approved company messaging.

5.1 Closing Point

The presentation should end by making the throughline explicit: one inbound email became a qualified, correctly-staffed, correctly-priced, formatted, delivery-owner-approved proposal — with multiple points where a human decision genuinely changes what the system does next, not a purely linear pipeline.

5.2 Known Scope Boundary

Contract execution itself (e-signature) is tracked as a status, not performed by the platform — intentionally: electronic signature carries legal and audit-trail requirements that are a deliberate integration point for a dedicated e-signature provider, not something to rebuild. Win/Loss status, Estimate vs. Actual, and Cost of Bidding are currently logged manually, since no CRM or timesheet system is integrated yet to supply this data automatically. These are deliberate and disclosed scope boundaries, not gaps discovered during the demo.